



M362 Unit 1

UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



From simple to
complex

Unit
1

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1589 7

CONTENTS

1	Introduction	5
1.1	What this course will cover	6
1.2	The aims of this unit	6
2	A normal day	7
3	Why concurrency and distribution are important	9
3.1	Modelling the real world	9
3.2	Efficiency and service	11
3.3	Example of a distributed system – a supermarket IT system	13
4	Concurrent systems	15
4.1	Definitions and principles	15
4.2	Concurrent system configurations	16
5	Distributed systems	20
5.1	Definitions and principles	20
5.2	Communications in distributed systems	22
6	From simple systems to complex systems	26
6.1	Sequential systems	26
6.2	Concurrent (but not distributed) systems	27
6.3	Distributed systems	30
6.4	Mobile systems	32
6.5	Enterprise systems	33
7	Case study	36
8	Summary	38
	Glossary	39
	References	42
	Acknowledgements	42
	Index	43

M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

This course is about concurrent and distributed computer systems – how they work and how to develop software for such systems. Although you may not have thought about it in these terms, we make use of concurrent and distributed systems every time we start up a normal personal computer and connect to the internet.

Concurrent computer systems can, in some sense, do several things at once.

At least, they appear to be doing several things at once. Some systems really can carry out several activities at the same time. However, other systems may just be juggling several activities very quickly, attending to each one in turn, whilst only doing one thing at a time. This is a bit like the magic of moving pictures on TV or in the cinema; you think you are seeing a real moving object, but it is really just a series of static images ('frames'), played fast enough to give your brain the illusion of movement. The illusion breaks down if the frames are gradually slowed down – first the picture looks a little jerky, then eventually you see the individual frames. In this course, we will 'slow down' the illusion of concurrent computer systems and see how they work in more detail.

A web server is a common example of a concurrent system – such servers respond to requests sent across the internet for particular web pages. Thousands of people may be visiting a popular website at the same time. It would not remain a very popular site if each person's request had to queue up until the previous person had finished.

Distributed computer systems have a number of component parts located in distinct places. The component parts are usually computer systems in their own right. These distinct places could be separate rooms or floors of the same building, or separate buildings of the same university, or they could be on different continents, or perhaps even on different planets – computer systems have been to Mars and other planets already.

The components of a distributed system are connected by some sort of computer network that enables them to communicate with one another. The internet is the best-known example of a distributed computer system.

Some distributed computer systems have **mobile** components that are not fixed in their location. Their communications with the rest of the system may take place by wireless technologies such as radio. Mobile phone networks are a prominent example of such a system – the mobile phones (or handsets) are the most visible part of a much larger system including radio antennae and fixed location computer systems for routing telephone calls and administering the system. Mobile components need not be wireless, though clearly some are more mobile than others. For example, a laptop computer is mobile whether its link to the rest of a networked system uses a wireless connection or a detachable wired connection.

In this course we will look at the benefits and the problems of concurrency and distribution in computer systems. To put these in context we will pay some attention to the historical development of these features. This development broadly shows a progression from simple systems to more complex systems. This is punctuated by occasional jumps back to simplicity, followed by a new cycle of gradual accumulation of complexity. For example, the personal computer invented in the late 1970s was initially relatively simple in computing terms. However, it has now acquired all the complexity of concurrency and distribution that characterised the much larger mainframe systems that preceded it. A similar progression is occurring at the time of writing with mobile phones

A **PDA (personal digital assistant)** is a small handheld computer.

and PDAs – manufacturers are continually adding new functions requiring more complex supporting hardware and software.

1.1 What this course will cover

M362 is divided into three blocks, each comprising a number of units.

In Block 1 (including this unit) we will concentrate on the study of concurrency. We will look at the concept of a process and how processes communicate. To illustrate important issues, we will use practical examples that involve the use of Java's threading mechanism and Java's concurrency mechanism.

In Block 2 we will consider distribution issues, including both theory and practice. Here we will be using the Java Platform, Enterprise Edition (Java EE) as an example of a modern framework for large-scale concurrent and distributed systems. Distributed systems consist of a number of different parts, each with their own functionality and their own set of problems. We will consider the various parts in detail and explain the techniques used in each area; furthermore we will study how the communication between the various parts of a distributed system is organised.

In Block 3 we will discuss some of the security concerns that are important to distributed systems. We will also look at heterogeneous systems and will conclude by giving an overview of recent developments in the area of mobile computing.

A more detailed summary of the contents of each unit can be found in the *Course Guide*.

We will also make use of a number of scenarios relating to a case study (a musical instrument internet auction site). The scenarios are steadily developed from very simple scenarios through to complex scenarios by the end of the course. We hope this will make the sometimes rather abstract ideas more real. This will also allow you to engage with some practical activities to show how such systems can be developed.

1.2 The aims of this unit

In this unit, we will explain the following:

- ▶ why concurrent and distributed computer systems are important (Sections 2 and 3);
- ▶ the historical context leading to modern concurrent and distributed systems (Sections 3, 4, 5 and 6);
- ▶ what is meant by concurrent and distributed systems (Sections 4 and 5);
- ▶ some of the ways in which these systems can be implemented (Section 6);
- ▶ the case study that we will use throughout the course to illustrate concepts in more detail (Section 7).

This unit mainly involves reading together with some exercises. There are also a couple of practical activities, related to the case study, to get you started with the Java development environment that you will be using throughout the course.

2

A normal day

It is a normal day – you sit down at a personal computer system to carry out some activities.

First you start up the computer system and it goes through some sort of starting sequence of activities such as loading the operating system and loading security software such as anti-virus software and a firewall. You start a word processor to create a report and then interrupt your writing to start up a spreadsheet from which to extract some figures to go into the report. You decide some additional information is needed that can be found on the Web, so you connect to the internet and run a web browser to search for this information. You find something relevant in a large downloadable file – you start downloading the file and go back to writing the report.

You realise you need some other information so you start an email program and send a message to a friend asking if they can supply the information you need. While waiting for the reply you start up a game you have seen on a website. The firewall software pops up a window to warn you that the website is attempting to plant some suspicious files on your computer, so you give the game a miss.

By now, the file has finished downloading, your friend has supplied the missing information and you can carry on with the report. While you are completing this, the anti-virus software automatically connects to a special website to download the latest virus definition files to keep your anti-virus protection up to date with any recently released viruses. You finish the report, save it on your computer and start printing out a copy on your printer. While it is printing, you also email a copy of the report to your colleague so that he can send you comments.

Exercise 1

Assuming that the personal computer has just one **central processing unit (CPU)**, can you identify examples of concurrency and distribution in the above everyday scenario?

Discussion.....

Until the operating system has loaded, a personal computer with one CPU can only do one thing at a time. After that, modern operating systems allow many activities to happen concurrently – only some of which are apparent to the average user. Indeed many of the examples of concurrency and distribution in this typical set of work tasks we may take for granted.

In the above scenario, there are a number of examples of starting one task, and putting it on hold while starting another task, for example when pausing use of the word processor to use the spreadsheet. In this case the system must keep track of what was happening with the word processor activity, so that it can be resumed, but it does not actually seem concurrent – we know that the system is not actively progressing two things at once. This is a sort of slow-motion concurrency where the user decides which activity is to proceed next.

In other cases, activities are clearly concurrent – for example, downloading a large file or updating the anti-virus software while editing a document with the word processor – the system really is continuing to download files as you edit the document. The other examples of concurrent activities are printing a document while using an email program, or the firewall popping up a warning while the user tries to access a dubious game

Some modern processor chips are **multi-core** – they have two or more processing units on one chip.

website. In all these cases, the concurrency is apparent rather than real, since there is only one processor being shared among all these activities. They differ from the examples in the previous paragraph in that the allocation of processor time to each activity in turn is controlled by the operating system once each separate activity has been initiated.

Aspects of distribution occur when interacting over the internet with other computer systems. For example, sending or receiving email requires communication with email servers and possibly many intermediate computers that forward mail or other messages. Similarly, downloading the file or the anti-virus update involves communication with remote computer systems. Distributed systems are always potentially concurrent and this is real (rather than apparent) concurrency – the email server is running at the same time as the various activities on your PC.

3

Why concurrency and distribution are important

There are several important motivations for employing concurrency and distribution in computer systems. It is very convenient to use concurrency and distribution when modelling aspects of the real world, which can itself be viewed as a concurrent and distributed system. A more technical reason is to ensure efficient use or sharing of resources such as hardware, software, data, or communications links. Concurrency may enable computer systems to respond more quickly to users, and distribution can make systems accessible from a wider range of locations. In other words they enable an improvement in the service to users. Any particular concurrent or distributed system will typically be a result of some or all of these motivations.

3.1 Modelling the real world

A familiar example of a concurrent and distributed system is the Earth itself, including all the animate and inanimate objects it supports. Consider some of the many layered and interrelated systems that exist. The geological structure of the Earth's surface is based on giant continent-carrying tectonic plates that move (very slowly), leading to earthquakes and volcanoes. There are other physical systems such as the climatic systems, including the systems of oceans and the atmosphere, and of course the planet interacts with the rest of the Universe, principally the Sun and our moon. On Earth the biosphere of living things, such as all the plants and animals, is layered on the underlying physical structure in vast complexity. At another level, we humans have built, and continue to develop, many systems as diverse as agriculture, ebusiness and space travel.

At any given time, in all of these systems there are a lot of things going on simultaneously, thus there is lots of concurrency.

The components of the Earth system, such as all the plants and animals, are distributed across a large number of distinct locations. So the Earth is also a distributed system, many components of which are clearly mobile, although continents, for example, move much more slowly than, say, cheetahs. Communication between the components of this system takes a variety of forms, ranging from sophisticated human and animal communications though to basic interactions (like one component eating another component!). The Earth and all its inhabitants constitute an immensely complex system, with a whole hierarchy of subsystems and sub-subsystems and so on.

Returning to computer systems again – the purpose of many computer systems is to model, monitor, influence or control a very small part of the huge complexity of systems on Earth. These may be natural systems, such as the Earth's climate, or they may be human creations, such as business or governmental systems. Most of the systems that we wish to model or control will be concurrent – several things will be happening at once – and distributed – things will be happening in different places but communicating in some way. So it is very natural to model or attempt to regulate these systems using computer systems that are also concurrent or distributed.

We should add a word of warning here that computer systems do not always reflect all the characteristics of the real world that they interact with or model. For example, some cities or areas have computerised road traffic systems that monitor and manage traffic flow by means of cameras, sensors, speed limits, traffic lights and so on. The designers

of such systems will typically simulate the behaviour of the system using a concurrent program running entirely on a powerful centralised system. By contrast, the system that actually carries out the monitoring and control is likely to have many networked processors distributed throughout the system. Processors based at each road junction or set of traffic lights carry out basic functions of data collection and control, but also coordinate, via a network, with the rest of the system. As we consider a number of examples during the course, the issues of concurrency and distribution should become clearer.

Real-time systems

An important class of computer systems comprises those that monitor and control other equipment or devices. Many of these are **embedded systems**: computer systems which form part of a larger system, such as a car, a printer, medical equipment, a spacecraft or the traffic management system we discussed above. Typically they have to monitor and control many devices at once, some of which may need to be given attention at well-defined time intervals in order for the larger system to function properly.

For example, many modern aircraft (Figure 1) are operated using a fly-by-wire system, where the aircraft controls are mediated by the on-board computer systems – an **unresponsive** computer control system could cause the aircraft to stall or even crash. Such computer systems that must guarantee to respond correctly to inputs within a specified time period are known as **real-time systems**. Concurrent operation is normally essential for all but the simplest real-time systems in order that the embedded system can respond in timely fashion when many demands arrive within a short time from different parts of the system.



Figure 1 Flight deck of the Airbus A320 fly-by-wire passenger aircraft

Embedded systems may also be distributed, with a number of distinct computers, each one controlling a part of the system. For example, a typical modern car may have dozens of microprocessors connected by a network, such as a CAN (controller area network), that together control everything from the braking system to the air conditioning.

3.2 Efficiency and service

There are also technical reasons why concurrency and distribution should be considered in computer systems. These normally relate to the efficient use of computing resources or of network communications. This is clearly important for financial reasons but also in maximising the service available to users from a given system. Historically, these technical reasons were the main driving factors behind early developments in concurrency and distribution. For distributed systems, an additional motivation is the need to share resources such as data across a number of locations, as shown, for example, by the explosive success of the internet and the Web since the mid-1990s.

Concurrency

Modern computer and communications systems are much more powerful than older systems. For example, current mobile phones have processors and memories that are much faster, and have a much greater capacity, than those found in many large computers of earlier eras (see Table 1).

Table 1 Typical resource capacities of systems from the 1960s to the present

Type of system	Date	Typical memory size – in kilobytes (KB) or megabytes (MB)	Processor speed (MIPS)
IBM 360/30 Mainframe	1965	32 KB	< 1
HP 3000 Mini-computer (small mainframe)	1975	128 KB	1
IBM PC	1981	64 KB	0.5
Apple Mac	1984	128 KB	0.7
Pentium 4 PC	2001	256 MB	2500
Mobile phone	2006	64 MB	400

MIPS means millions of instructions per second.

Processor speeds in Table 1 must be regarded as only approximate since the exact MIPS rating is affected by many different factors and the exact definition has changed somewhat over the years. However, the table does show the huge increases in both CPU speed and typical memory size.

The resource demands of users and applications have, however, also increased enormously. Current mobile phones are more than capable of handling the voice calls and text messages that were the only functions available on early mobile phones. More demanding applications, such as picture messaging and online gaming, continue to increase the requirements on the limited hardware and communications resources. This means that efficiency is still important, especially in making interactive systems **responsive** to users.

Early computer systems were **centralised** – all, or almost all, the processing was carried out at one central location, a so-called **monolithic architecture**. Typically the great central computer system was the size of a wardrobe and was attended by white-coated operators in a large room with extensive air conditioning and water-cooling. Access to such systems was initially via input and output devices in the same room as the computer processor (see Figure 2).



Figure 2 IBM 360 series mainframe from 1965

Later developments allowed access from remote computer **terminals** (usually so-called **dumb terminals**), which had very little local processing power and were basically a screen together with a keyboard and some communications links. These terminals were linked to the central computer system by local cabling or, if more remote, via the telephone system using a modem.

By today's standards early computers were very large, slow and expensive. However, their processing units could calculate and manipulate data much faster than their supporting devices (like hard disks, tapes or printers) and enormously more quickly than their human users. It often made sense to employ concurrency so that a computer system could progress several activities at once, while waiting for slower input or output devices to complete some task. Similarly, a large computer system could employ concurrency to service a large number of human users at the same time. Each user was given the impression that the computer was responding just to their request, whereas in practice the computer was sharing its time between all the users without them noticing a significant delay, a technique known as **time-sharing**.

Time-sharing in chess

Expert chess players sometimes take part in events where they play games against a number of other less expert players at the same time – in chess jargon this is known as a *simul* (short for simultaneous). There may be typically about ten players, each one playing a game where the single expert is their opponent. The expert moves about from one game to another, giving each one their attention for a short time, making a move and then moving on to the next game. This is similar to the way a time-sharing computer system appears to be interacting with many users at once.

If a chess simul is carried out over the internet, for example, then in principle, it is not possible for the non-experts to tell that there is only one expert. Chess playing computers are now so effective and powerful that they can beat the best human experts (chess grandmasters). We could replace the human expert in an internet chess simul event with a chess computer acting concurrently and, again, the human players might well be unable to tell.

Distribution

One of the early motivations for work on distributed systems was to allow remote access to powerful computer systems capable of carrying out complex scientific calculations. This was one of the reasons for development in the late 1960s of the system of linked computers in the USA, known as ARPANET. This system eventually spread across the world and developed into the internet.

In the early days of ARPANET, it soon became apparent that these links provided a ready means for remote users to exchange files and messages – file transfer and email are still among the most popular uses of the internet.

Initially this activity was seen in terms of linking existing separate systems, but in time such systems came to be viewed as, or even designed as, one distributed system with component computer systems. This gave a number of additional technical reasons for designing systems as distributed rather than centralised. In a monolithic system, the entire processing load falls on the single central processing system, and in cases of heavy loading this can give rise to poor performance. Also, if using a system of dumb terminals, users can make no progress if the communications link between the terminal and the central computer fails. In a distributed system, much of the system data and processing power can be located close to the users. This reduces the loading and reliance on limited or potentially unreliable communications links. The local parts of a distributed system may still be able to function in a limited way even if some communications links fail – for example, you can still use a PC to carry out tasks even if it temporarily loses its network connection.

The first message on ARPANET was sent on 29 October 1969

3.3 Example of a distributed system – a supermarket IT system

Large supermarket companies in the UK have hundreds of stores. It would be very difficult (though not impossible) to handle all their IT requirements from one central location, and would necessitate transfer of large quantities of data between stores and the centre. By locating processing power and detailed data within the stores, this undesirable situation can be avoided.

Each large supermarket company in the UK has a distributed computer system with typical operating costs of hundreds of millions of pounds per year. Every store has its own computer system, linked to the sales terminals, keeping track of what goods are

sold and what is in stock. The store computer system is linked to central systems at a 'head office' that help to coordinate activities such as deliveries of goods and maintenance of financial and other information for the whole company. At the same time, all the warehouses and distribution centres will use local computer systems to manage their local activities and to communicate with the head office systems.

Decentralisation also makes it easier to extend the system when a new store is opened – the new local computer systems provide the extra processing and data storage likely to be required. Such a system is **scalable** – it can be scaled up or down to meet the likely demand. On the other hand, for the supermarket company to function efficiently as one organisation there must be a degree of central coordination of many aspects such as finance, pricing, warehousing, distribution and so on. Since immediate processing is mostly handled at local level, much of the communication with head office can take place at night when stores are closed or quieter and the network is likely to be less busy. The balance between centralised and local resources is one of the key design challenges in distributed systems.

Exercise 2

Would you say that the distributed approach to supermarket IT systems is employed for modelling reasons or for efficiency reasons?

Discussion.....

It is likely to be a mixture of both. Locating a computer system in every store and warehouse, as well as at the head office, ensures that the structure of the overall system matches the geographical structure of the supermarket company and should make it more straightforward to adapt the IT systems as new stores open, or others close down, in response to demand. It also allows stores some autonomy to adjust their operations to local conditions where appropriate. There are efficiencies in undertaking as much processing as possible locally to each store or warehouse, thus limiting the amount of data communication needed and allowing the data communication to happen at quieter times (such as overnight).

4

Concurrent systems

We have discussed a number of examples of concurrent systems, but so far we have deliberately been a little vague about what we mean by a concurrent system. In this section, we will make the concept more precise and we will also discuss the various ways in which concurrency can be implemented on computer systems.

4.1 Definitions and principles

For a computer system to exhibit concurrency there must be a number of distinct activities to be carried out. In a **sequential** system one activity must finish before the next activity can start (Figure 3).

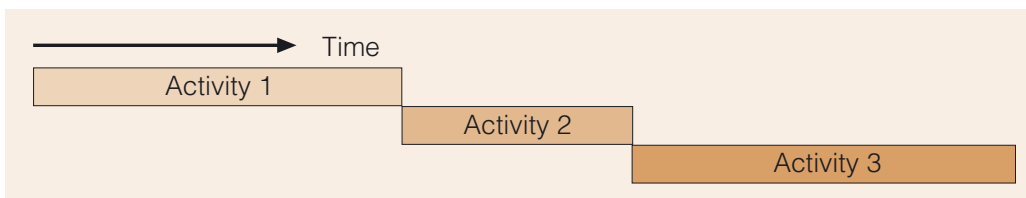


Figure 3 A sequential system – three activities completing in sequence on one processor

In a concurrent system there is the possibility that, at any given time, several activities are in progress – that is, somewhere between their respective starting and ending points. Concurrency can occur in a number of different ways.

A system that allows several activities to be actually executing at the same time is known as a **parallel** system. A parallel computer system requires more than one processor, each one carrying out one of the activities at a given time (Figure 4).

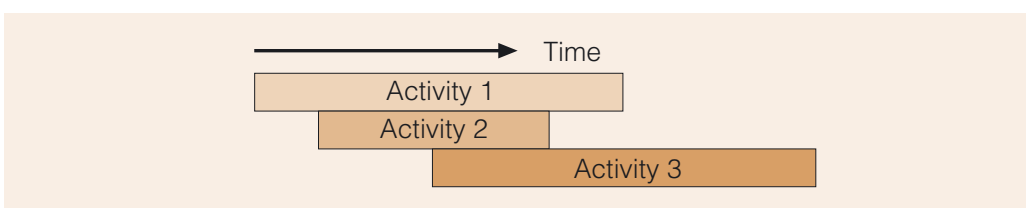


Figure 4 A parallel system – three activities executing at the same time on three processors

Many concurrent systems are not truly parallel – they give the impression of parallel execution by sharing the resources of a single processor between a number of activities. This is sometimes called a **pseudo-parallel** system, meaning ‘not-quite-real’ parallelism. In this case, the processor is allocated to each activity in turn for a short period of time, so that each activity can make some progress without waiting for any of the others to complete. Figure 5 shows an example of how the execution time on a single processor might be allocated to parts of each of three activities in turn.

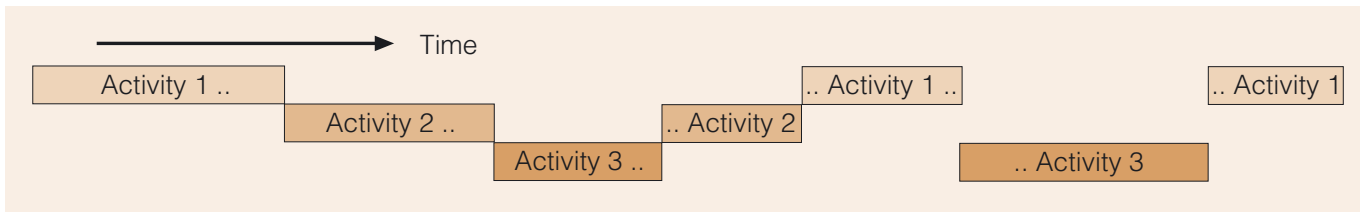


Figure 5 A pseudo-parallel system – three activities sharing execution time on one processor

So we can characterise a concurrent system as one that has the *potential* for parallelism. It may or may not actually execute in parallel, but it is designed in such a way that it could do so if parallel resources are available.

As we have seen, a concurrent approach may be motivated by either modelling convenience or efficiency (or indeed both reasons together). If raw efficiency is a primary reason, such as in large-scale scientific calculations like weather forecasting, then achieving actual parallelism is likely to be important in maximising performance. If service and responsiveness to users is the most important issue, as in interactive systems serving many users, even pseudo-parallel operation may greatly improve the perceived performance for each user.

For example, a web server will be much faster in its average response if it can switch rapidly among many users than if it has to complete dealing with one user before addressing any later requests. Of course it will also normally operate faster if it has more processors available to share the load, but this may or may not be the crucial factor affecting perceived response times.

Computer systems can occur in a very wide variety of configurations, but for our present purposes we can distinguish a small number of important subsystems. Each system has one or more **processors** (or CPUs) that carry out the calculation and data manipulation functions. Computer systems also have a variety of means of storing information. Short-term retention of information is the role of **memory**, which may, for example, take the form of semiconductor **RAM (random access memory)**. Long-term information retention is by means of **storage**, such as magnetic or optical disks.

4.2 Concurrent system configurations

Burns and Wellings (1998) identify three possible basic configurations that apply in concurrent systems (Figure 6).

- ▶ A number of activities share a single processor – a **uniprocessor** system.
- ▶ Each activity has its own processor on a centralised system – a **multiprocessor** system.
- ▶ Each activity has its own processor and the processors are distributed – a distributed system. (The separate components may be linked by a local area network (LAN) or a wide area network (WAN).)

In practice, things may be more complex – each processor on a multiprocessor or distributed system may also be shared between several activities.

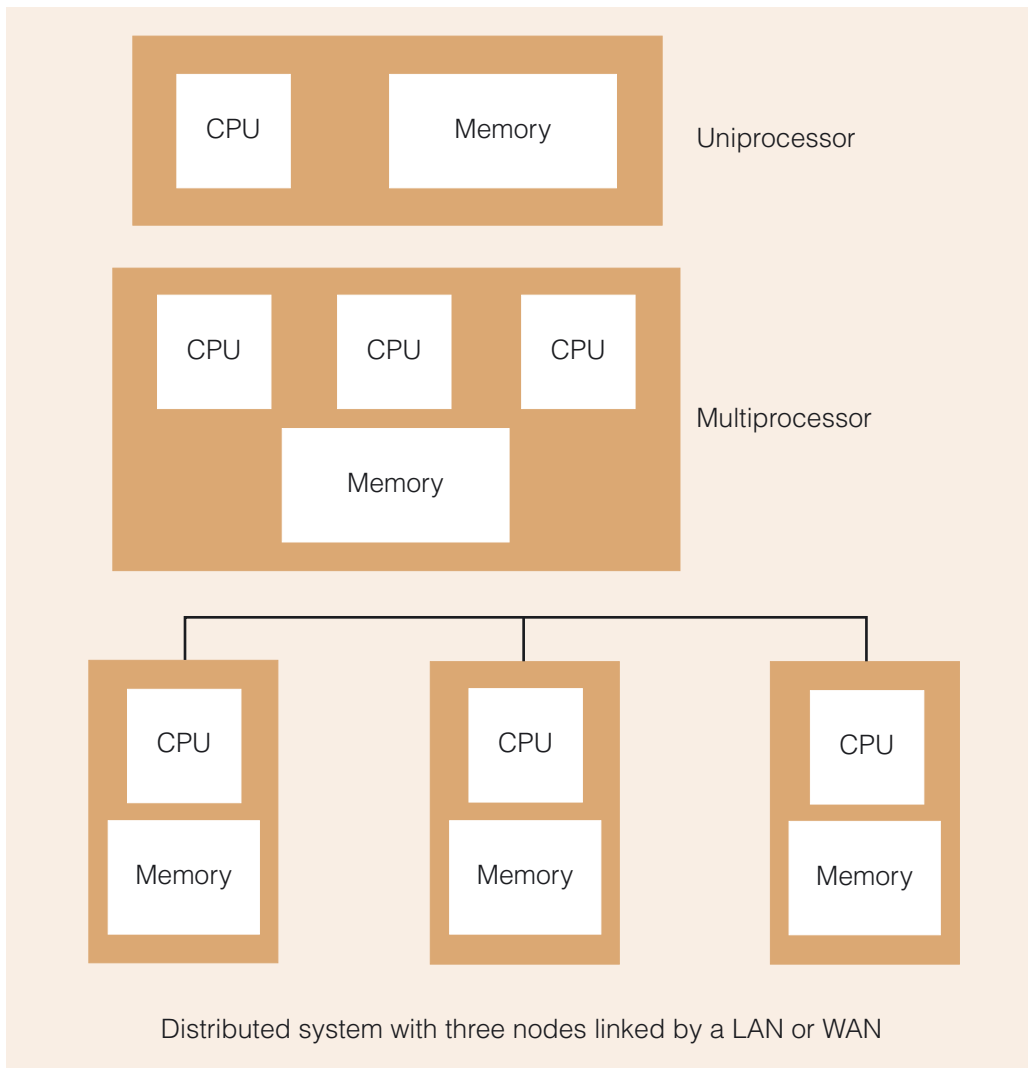


Figure 6 Possible configurations of concurrent systems

Whether a system is parallel or only pseudo-parallel, there are a number of important issues to consider in terms of how the software operates, for example:

- ▶ how the activities are specified;
- ▶ how the activities share the processor (or processors);
- ▶ how the activities share memory;
- ▶ how the activities communicate with each other or with any controlling software.

It turns out that most of these issues can be dealt with in similar ways, regardless of whether the system is truly parallel. Therefore, the key notion for software developers is concurrency – properly designed concurrent software should function correctly, irrespective of whether the execution environment is truly parallel or merely pseudo-parallel.

It is often convenient to separate the **application software** for the activities from the software that controls the resource sharing and communication between activities. This normally leads to the use of an **operating system** that supports concurrency as most modern operating systems do. In later units, we will explore the possible division of labour in concurrent software between the operating system, the programming language and other implementation frameworks.

SAQ 1

Which of the following system configurations are capable of being concurrent, parallel, or neither?

- (a) One processor, three activities
- (b) Three processors, two activities
- (c) Three processors, five activities
- (d) Two processors, one activity

ANSWER.....

- (a) Concurrent, but not parallel – the activities can share one processor in a pseudo-parallel fashion.
- (b) Parallel and therefore concurrent – each activity can progress simultaneously on one processor, with one idle.
- (c) A mixture of parallel and pseudo-parallel activity is possible here – some processes must share one or more processors.
- (d) No concurrency or parallelism is possible as there is only one activity – this can run on one of the processors while the other remains idle. If the activity could be split up into two or more subactivities then some concurrency and even parallelism would be possible.

We should also point out that concurrency or parallelism may be present at a variety of levels within a computer system. The individual hardware components of a system, such as the CPU, the memory and the storage, can typically all operate to some extent in parallel by partially overlapping their activities. On a modern CPU chip there may be more than one processing unit (referred to as a 'core' or 'execution core'), and even these processing units normally have internal components that can operate in parallel in order to speed up execution.

If there are two processing units per chip this is known as dual-core technology – the term multi-core technology is used if there are more than two processing units per chip. It is even possible to have multi-core multiprocessor systems, where several processors are connected (as in Figure 6), and each processor is multi-core. In this course we will not focus on this low-level, mainly hardware-based, parallelism, but it is useful to be aware that it is present.

The transputer

Introduced in 1984, the transputer was a revolutionary development because it combined processor, memory and communications links in one chip. The communications links made it easy to connect a transputer to up to four adjacent transputers, in building-block fashion. The aim was to make it easy to build large arrays of transputers operating in parallel. This was an innovative alternative to conventional multiprocessors where the connections were typically much more complex, expensive and less scalable. Figure 7 shows a circuit board with an array of 42 transputers – in some designs, many such boards were combined to construct powerful parallel computers with hundreds or even thousands of processors.

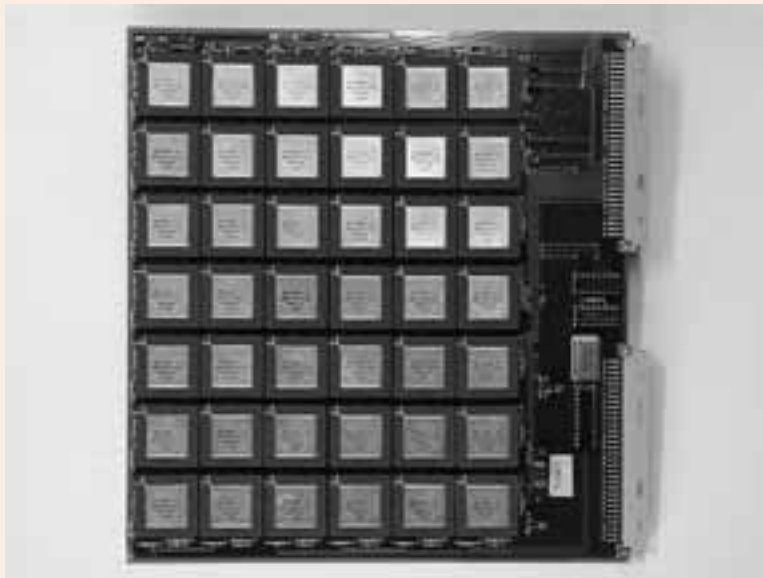


Figure 7 An array of 42 transputer chips

The transputer was a short-lived success, attracting great interest for its novel hardware design and its equally novel programming language, occam. It was eventually superseded by performance advances in more conventional processor architectures. It was arguably a little ahead of its time (and too expensive) at first and in the later 1980s failed to keep up in price/performance terms with alternative computer chip designs.

5

Distributed systems

A distributed system, where processing components occupy a number of different locations, may be contrasted with a centralised system, where all or most of the processing power resides in one location (even though there may be some form of remote access). We have cited the internet as an example of a distributed system. In this section, we will make the concept of a distributed system more precise and we will also discuss the various ways in which distribution can be implemented in computer systems.

5.1

Definitions and principles

Distributed computer systems have been defined in a number of different ways. It is generally agreed that a distributed system contains a number of distinct components at different locations, where each component is, in some sense, a computer system itself. These distinct components can communicate with one another via some sort of network, although there are a wide variety of technologies available for such communication. It is also sometimes suggested that a distributed system should appear to its users as a single coherent system providing a range of functions in such a way that the user may not know, or even need to know, that the system is distributed. This last requirement is called **transparency** and holds to varying degrees for systems that are generally agreed to be distributed systems.

SAQ 2

Does the internet qualify as a distributed system according to the definition above?

ANSWER.....

The internet certainly satisfies the first two requirements above. Its components are computer systems – either individual computers, like PCs and web servers, or computer networks, depending on how detailed a view you take. These components are connected by computer networking and communicate using internet protocols such as TCP/IP. It is harder to say if the internet is a coherent, transparent system providing a range of functions – to naive users it may well be transparent, especially on a fast connection. At any rate, the internet is generally agreed to be a distributed system, so it must be the case!

Transmission Control Protocol/Internet Protocol (TCP/IP) are the two protocols that form the basis of network communication. See also Subsection 5.2.

Coulouris et al. (2001) list the significant characteristics of distributed systems as concurrency, the possibility of independent failures and the lack of a global clock.

- ▶ Distributed systems are necessarily concurrent, in that each host (or node) can operate independently and potentially in parallel with other hosts.
- ▶ It is also clearly possible that one or more hosts may fail, possibly without other parts of the system becoming aware of this – hence introducing different sorts of problems from those in non-distributed systems.
- ▶ The issue of time in distributed systems is dealt with in a later unit, but the basic idea is as follows. Each host typically maintains its own value of the current time. Even if all hosts at some initial point had precisely the same value of the current time, over the longer term the equipment that keeps track of time can drift slightly so that the hosts are no longer synchronised.

Bacon (1998) extends the previous list of characteristics as follows.

- ▶ Distributed systems are also characterised by the presence of communications delays (unlike, say, a multiprocessor system) – this is one of the factors leading to the impossibility of synchronising each host to exactly the same value of the current time.
- ▶ Distributed systems have the potential to enter an inconsistent overall state (the issue of system state will be considered in a later unit).

The individual components of a distributed system that have some processing capability are known by a variety of names including **nodes** or **hosts** – with host being the most commonly used term. These hosts can take many forms, ranging from large mainframe systems through servers and standard personal computers to mobile phones, PDAs, or even embedded systems. Systems that have centralised processing but have limited remote access, for example dumb terminals with little local processing power, are not normally considered to be distributed systems – there must be some distribution of processing capability for a system to merit this classification.

Some key points in the history of the internet

1968 Packet-switching network at NPL

The National Physical Laboratory in the UK sets up a network that communicates using a technique whereby data is chunked up into smaller units – so called data packets.

1969 ARPANET (Advanced Research Projects Agency Network)

The ARPANET packet-switching network is set up at four US universities with US Department of Defense funding to explore the idea of a resilient network in case of major war. (In packet switching, data packets travel in a series of steps, from one computer to another via a number of intermediate computers or other routing devices.) By 1971 there are 15 nodes and by 1972 this has grown to 37 nodes. This is generally agreed to be the forerunner of the internet.

1971 Email and FTP on ARPANET

Email more or less as we know it (that is, email using the *name@host* format) is introduced to ARPANET. About the same time the File Transfer Protocol (FTP) is developed to do what the name says – provide a reliable means of transferring files between ARPANET computers. It is still in widespread use today in the background of many activities on the internet.

1974 The word ‘internet’

The term internet is first used by Vint Cerf and Bob Kahn in a paper on Transmission Control Protocol (TCP).

1983 The internet becomes official

The Internet Activities Board (IAB) is created; the TCP/IP network protocol replaces older protocols used on ARPANET and elsewhere.

1990 The Web begins

Tim Berners-Lee implements a hypertext system at CERN in Geneva (it was to be publicly released two years later as the World Wide Web).

1993 First graphical web browser

Mosaic, the first graphical web browser is launched by Marc Andreessen, leading to explosive growth in the number of internet hosts and users of the Web – and the rest is history!

Sources: Living Internet (2006), Sterling (1996)



Internet history.



Email history.

5.2 Communications in distributed systems

Our definition of distributed systems stated that the individual hosts are connected by some sort of communications network. Before we look at this more closely, we need to clarify how the communication in a multiprocessor system differs from that in a distributed system. The CPUs in a multiprocessor system are located very closely together, normally within the same box, and are typically linked by some sort of parallel wiring connections (known as a **parallel bus**) that operate at high speed. For a system to be regarded as distributed, the communications must be rather less closely coupled than this, and the normal mode of communication is by passing messages in some agreed format along the communication link.

It is possible to connect two computers directly by wires or perhaps wirelessly (e.g. via infra-red or Bluetooth), for the purpose of exchanging data. For example, uploading pictures from a mobile phone, or a digital camera, to a desktop computer would fall into this category of activity. This could be regarded as a very simple sort of distributed system, but this is right at the limits of the definition. We normally mean a more complex system with a number of independent computer systems collaborating on a shared task.

The communication in distributed systems is organised as a system of layers, from the very low level where we specify the cables and other physical connections (the physical layer), to layers at a higher level. These higher levels specify how computers are to interpret the messages they receive. Here, we will briefly discuss the lower-level communications layers and then look at some of the higher-level communication protocols.

Low-level connections

At the lowest level of computer communication we are dealing with streams of bits. In order to send and receive these bits, there are a number of different technologies that cover a wide range of possible communication speeds.

One way of linking computers within a building or in adjacent buildings is a **local area network (LAN)**, using technology such as Ethernet. The physical links might be via copper wires, fibre optic connections or wireless connections (usually by radio). Ethernet LANs were originally 10 Mbps (10 megabits per second) but are now typically 100 Mbps (Fast Ethernet) or even 1000 Mbps (Gigabit Ethernet). The fastest versions of Ethernet normally use fibre optic cables and are used for so-called ‘backbone connections’ – linking smaller networks that use copper wires or wireless connections internally.

Ethernet can also run using wireless technology (radio links) – the IEEE 802.11b standard for this (known as Wi-Fi) defines a maximum rate of 11 Mbps, although actual typical data transfer rates are significantly less (about half) due to overheads in the communication protocol. Although slower than most wired Ethernet, it is fast enough for many network applications. A newer version of the Wi-Fi standard (IEEE 802.11g) defines a raw data transfer rate of 54 Mbps and this has been widely adopted by manufacturers in recent years. An even higher maximum data rate (of up to 540 Mbps) is specified by the IEEE 802.11n version of the standard.

A **wide area network (WAN)** may be used to connect components of distributed systems where the components are more distant than those served by LANs. These normally use either shared public telephone connections or private data connections. Communication may be via wires, optical fibres, radio or microwaves, possibly using satellites. Data rates for WANs are typically lower than those achievable for LANs.

A comparison of a number of wired communication technologies, in terms of their typical data transmission rates, is shown in Table 2.

The layered approach for communication in distributed systems will also be discussed in *Unit 5*.

Data transmission rates are measured in bits per second (bps), usually in units of kilobits per second (Kbps) or megabits per second (Mbps).

Table 2 Wired communication technologies

Technology	Speed (bps)	Medium	Use
Plain old telephone service (POTS)	56 K	Copper wires	Domestic phone calls
Digital subscriber line services (DSL)	8 M	Copper wires	Home broadband link
Ethernet	10 M	Copper wire/fibre optic	Local area networks
Fast Ethernet	100 M	Copper wire/fibre optic	Local area networks
Gigabit Ethernet	1000 M	Fibre optic cable	LAN backbones
Asynchronous transfer mode (ATM)	622 M	Fibre optic cable	Wide area networks

Historically, organisations wishing to interconnect their distant computer systems had to build, buy or rent their own long-distance links. Nowadays, many such connections may use the internet if reliability and security considerations permit.

Distributed systems may also include mobile components such as wired or wireless laptops as well as mobile phones with suitable long-range radio communications. The common communication standard for mobile phones in Europe and much of the rest of the world is GSM (Global System for Mobile communications). GSM data rates are typically much lower than for the long-distance links provided by WANs, but continual developments in mobile phone standards (such as GPRS and UMTS) are greatly increasing the potential data rates. Table 3 shows a comparison of wireless communication technologies.

Table 3 Wireless communication technologies

Technology	Speed (bps)	Range	Medium	Use
IrDA	72 K – 4 M	0.2 – 1 m	Infra-red	Personal area networks – very short-range connection of devices such as phones, PDAs or laptops
Bluetooth	3 M	1 – 100 m	Radio	Short-range connection of devices such as phones or PDAs
Global System for Mobile communications (GSM)	14.4 K	10 km	Radio	2G mobile phones
General Packet Radio Service (GPRS)	114 K	35 km	Radio	2.5G mobile phones
Universal Mobile Telecommunications System (UMTS)	2 M	2 km	Radio	3G mobile phones
Wi-Fi (IEEE 802.11b)	11 M	35 m	Radio	Wireless LAN
Wi-Fi (IEEE 802.11g)	54 M	25 m	Radio	Wireless LAN
Wi-Fi (IEEE 802.11n)	540 M	50 m	Radio	Wireless LAN
WiMAX (IEEE 802.16)	10 M	10 km	Microwave	Wireless WAN

Note that the speeds and ranges given are only typical values as implementations of these standards vary considerably. In addition, there is often a trade-off between range and data rates – the greater the range the slower the achievable data rate. As with any wireless technology, security is a concern, as wireless communication is more easily intercepted than wired communication. You are not expected to memorise the numeric values or the acronyms in Tables 2 and 3 – these will all change as technology develops – rather we want to demonstrate the wide range of technologies and communications capacities used for different applications.

Higher-level protocols

The internet is a vast network of networks, interconnecting millions of host computers and networks. The hosts and their software are heterogeneous – that is, they are of many different types and combinations: a variety of hosts, such as mainframes, PCs, Apple Macs or mobile phones, run a range of software, such as Windows, Linux, and other, vendor-specific, operating systems. The internet depends heavily on universally agreed communication protocols and other standards. For example, most communication across the internet uses the **TCP/IP** protocol which specifies the detailed format of messages that can be exchanged between hosts.

Messages (or data files) are sent across the internet in the form of so-called data packets. Each packet consists of some data together with a packet header, which contains details about the packet such as its size or the destination address. The packet header can be thought of as the envelope of a letter sent through a normal postal system with the data forming the content of the letter.

The TCP/IP protocol uses the information in the packet header to make the internet communication robust. For example, since multiple routes are available between any two hosts, it is possible that there may be some kind of failure or congestion on one route, in which case another route for messages will be found automatically. Also, any lost parts of a message will be sent again.

However, because the internet is open to the whole world, there are concerns about its reliability and security that may limit its usefulness for some applications. Security issues can be addressed to some extent by secure internet protocols such as Secure Sockets Layer (SSL), which encrypts the communication to ensure its privacy. We will return to the issue of secure protocols later in the course.

Some key points in the history of distributed systems

1971 Wireless systems

The first wireless computer network, ALOHAnet, is developed by researchers at the University of Hawaii to link the University's campuses, which are scattered across the Hawaiian Islands.

1973–1975 Ethernet

One of the most widely used technologies for local area networks, Ethernet, is invented at Xerox's Palo Alto Research Center (PARC). (Xerox PARC was also responsible for the creation or popularisation of many key aspects of modern computing such as object languages and graphical user interfaces.) Ethernet is now available in many flavours up to 1 gigabit per second speeds.

1981 Mobile computer systems

The Osborne-1 lays claim to being the earliest portable computer system. ('Luggable' or muscle-building might have been more appropriate terms, and it only ran on mains power.) A more realistic first laptop, the Grid Compass released in 1982, could at least run on batteries – this machine made it right out of this world on the NASA space shuttle, but at a cost of \$10 000 it was not a casual purchase.

1990 Digital mobile phones

The date of the first mobile phone depends on what you count as a mobile phone, but the first really portable (as opposed to car-based) digital phones are released in 1990 in the USA (cell phones) and in 1991 in Europe (mobile phones). These 2G or second-generation phones are, at the time of writing, only slowly being challenged by more capable, but more expensive, 3G phones.

1999 Distributed computing

The (relatively old) idea of harnessing large numbers of networked computers to provide massive computational power is, with the growth of the internet, attempted on a worldwide scale. Some of the best known projects of this type seek to analyse radio signal data for signs of extra-terrestrial life (SETI@home), or run models that help forecast the effects of climate change (climateprediction.net). These run in the background on thousands or even millions of PCs.

2001 Grid computing

Grid computing is a service for sharing the processing and data storage power of the internet, making computing power a utility rather like an electricity distribution grid. The initial applications are mainly driven by the needs of very large-scale scientific projects. At present the Grid is a 'work in progress' with major efforts around the world to provide the necessary software infrastructure.



Searching for extra-terrestrial life.



Modelling climate change.

6

From simple systems to complex systems

The current state of concurrent, distributed and mobile computer systems is the result of a long series of historical developments from earlier simpler systems. Studying their evolution is a useful educational approach to developing a detailed understanding of these complex systems. Here we will start by considering simple (i.e. sequential) systems and, by adding a layer of complexity/additional capability at a time, work through concurrent and distributed systems up to mobile systems. At each stage we will consider the reasons why such a development might be undertaken – the benefits – as well as the additional complications or problems that may arise as a result – the costs. We will end this section with a brief outline of what we mean by enterprise systems, as an example of concurrent distributed systems with possibly mobile aspects.

History, of course, is never that simple. It is famously said that it repeats itself and we shall see that, on several occasions, the history of computing systems has partially repeated itself.

6.1 Sequential systems

Motivation

The earliest electronic computers, such as ABC and ENIAC, were built in the 1940s. They were bulky and unreliable, needing continual maintenance to keep them running. A purely sequential computer system may never have existed but it is a fair approximation to the truth to say that early computer systems were designed to do one thing at a time. By 'one thing', in this case we mean the system was designed to undertake one main task at a time, usually corresponding to one program controlling the activity of the system.

We could view the building of these sequential systems as a way of engineers 'learning to walk before they could run'. There is plenty to think about in designing a computer system to do one thing at a time. Trying to do several things at once in a computer system introduces additional problems that must be solved before the system can work reliably.

Over time it was realised that there were advantages in having these large expensive pieces of hardware being more fully utilised by having them do several things at once. This led to a gradual increase in the complexity of the software that controlled the computer systems. Inevitably there was eventually a reaction to this.

Almost forty years later, when electronics had advanced enormously in miniaturisation with vastly reduced costs, it became feasible to design personal computers – computers that individuals could own, or at least use, just for themselves. This was a revolutionary concept when such computers first made an appearance around 1980; at this time most computers were large wardrobe-sized things owned by companies or universities. Early personal computers were largely sequential systems in software terms. It was assumed that they would be used by one person at a time to carry out one activity at a time. However, quite early personal computer systems did support a small amount of concurrency such as being able to print in the background while allowing the user to do something else.

Apart from a few early systems mainly used by hobbyists, the real takeoff for personal computer systems came with the early Apple computers and especially the IBM PC (in 1981). The IBM PC launched Microsoft on its way to global success when IBM contracted the, at that point small, software company to provide the operating system. The operating system Microsoft acquired (but did not actually write) was renamed MS-DOS – this was a single-user operating system that allowed the user to run one program at a time.

Benefits

The main benefit of a sequential system is simplicity of design and use. Programming is relatively straightforward for certain sorts of applications, since there is no need to think about how a number of concurrent activities can be organised to share the computer without interfering with each other. The instructions in the code of the sequential program are carried out in a well-defined order specified by that program and no other. If something goes wrong with a sequential program it is relatively easy to trace through the instructions one at a time, to find the cause of the problem. The operating system for a sequential system can also be relatively simple.

Costs

Different parts of the hardware of a system can operate at very different speeds – CPUs and RAM operate much faster than hard disks or printers, for example. A system that only does one thing at a time will always have bottlenecks in performance where it has to wait for its slower components.

These problems with hardware speeds can be partly overcome by means of relatively simple operating system software. For example, the MS-DOS system for early PCs allowed the system to print a document ‘in the background’ while the user continued to use the computer for another task. Even so, a mainly sequential system of this sort only permits one main activity at a time.

Early computer systems were physically very large, slow and hugely expensive by modern standards. Hence very few were built at first and it was difficult for potential users to ‘book time’ to use the computers. It therefore made sense to think about how this expensive computer hardware could be used to do more than one thing at a time – efficiency!

We have also mentioned that many systems in the real world are concurrent – to monitor and control these with a system that runs sequentially is not impossible but certainly much more difficult. Before the current widespread availability of efficient concurrent operating systems, designers of small embedded systems had to use programming ‘tricks’ in order to compensate for the use of essentially sequential software to model a concurrent world. So, sequential systems work quite well for simple systems that do not require concurrency but actually make software design more difficult when the system has to interact with a concurrent real world.

6.2 Concurrent (but not distributed) systems

Motivation

We have already seen that some concurrent running of hardware aspects of systems was developed at an early stage in computing history for efficiency reasons. Operating systems allowed systems to do things such as printing, or reading input from a slow device, at the same time as the system was doing something else with its faster components like the CPU or memory. It was a short step from this to the idea of allowing

several different activities (so-called ‘jobs’) to coexist and share the processor – one could proceed while others were waiting for slow input or output to complete, hence making more efficient use of valuable and scarce computing resources.

Figure 8 illustrates what happens in a system without concurrency that has one activity, labelled Job 1. The processor time is allocated to Job 1 until this job requires a slower activity (disk access). At this point Job 1 is paused and waits until the disk operation has completed before continuing. The time while Job 1 is waiting for the disk is completely wasted in that the processor is idle.

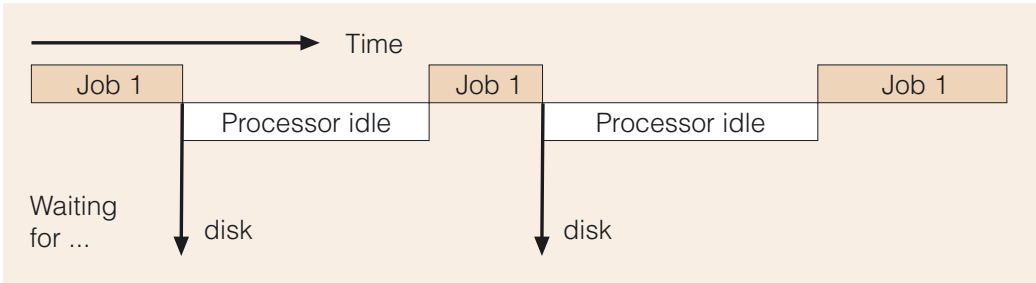


Figure 8 Sequential system with processor idle during disk accesses by a single job

Figure 9 illustrates how a concurrent system can handle three jobs. The processor time is allocated to Job 1 until it requires a slower activity (disk access). At this point Job 1 is paused and Job 2 executes until it needs to carry out some printing. Job 1 then resumes and so on in turn with each job executing until it reaches a requirement for a slower activity, at which point it gives up the processor to another job. This is a somewhat idealised representation of processor sharing and later units consider more realistic scenarios.

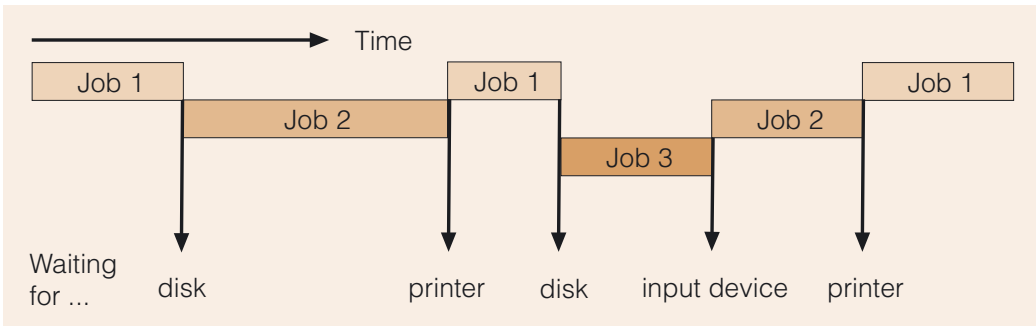


Figure 9 Concurrent system sharing processor time between a series of jobs

Large early computers were initially not at all interactive – they had one ‘console’ with a keyboard which one operator used to control the system (see Figure 2). Later systems had terminals that allowed many users to interact with the computer system. Clearly, on a sequential system this would lead to major user frustration, as the first person to start using the computer would claim exclusive use of the system until their job was finished. Hence a concurrent time-sharing system that shared the processor’s time between each of the terminals was required. The same sort of ideas that allowed several jobs to run concurrently could now be applied to allow each user to make progress and run a different program, with each one having the impression that the system was only serving them.

Of course this impression depended very much on the computer system being powerful enough to quickly rotate its attention among all the users – a heavily overloaded system or a faulty program could quickly destroy the illusion and cause the system to seem very slow and unresponsive. Figure 10 illustrates how this works for three users sharing the processor.

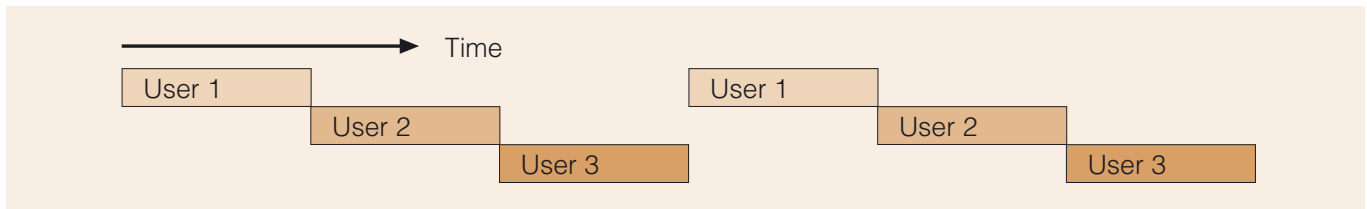


Figure 10 Concurrent system sharing processor time between a number of interactive users

This shows each user getting a fixed and equal amount of processor time – this is known as **time-slicing**. In practice, the allocation of the processor time, known as **scheduling**, is usually more complex – for example, allocation of time to the activities for each user may vary because of differing access to slower input and output devices as in Figure 9.

Early personal computers were only able to run one application program at a time because they had relatively simple operating systems. The past decades have seen a steady increase in the complexity of personal computer operating systems, partially repeating the history of mainframe computer operating systems in the 1960s. Modern personal computers have operating systems, such as Windows, Linux or Mac OS, that allow a high degree of concurrent operation.

Benefits

Development of concurrent systems allowed more efficient use of expensive hardware. It also made it possible for many users to interact with one large system, sharing its fast processor, so that each one seemed to have their own computer. These benefits still apply to large mainframe systems, widely used by large corporations such as banks to manage their head office operations, but are also relevant to more recent kinds of systems such as web servers.

In personal computers the emphasis in concurrency is to allow the user to carry out a number of tasks at once, or be able to interrupt one task temporarily to run another activity, and to make the system responsive to user requests even when it is executing a number of other activities.

The software design for many computer systems can be simplified by taking advantage of a concurrent environment. We gave earlier the example of real-time systems, where more complex systems are normally built and checked for timing behaviour by structuring the software as a set of cooperating concurrent activities. Some very large-scale scientific calculations, such as climate simulations or applications in pharmaceuticals and biology, are only feasible by using massively parallel systems (with many processors) to spread the computational load.

Costs

All these potential benefits come at the cost of requiring more complex operating systems or other software to manage many activities proceeding at the same time. More complex and expensive hardware, such as a multiprocessor system, may be required. The additional complexity of hardware and software introduces new ways for software to fail, possibly with subtle errors.

For example, if a system has a number of concurrent components, they may find themselves in competition for resources (several concurrent activities may want to use the printer or the network connection at the same time) and the operating system must resolve such competition safely. If this is not well handled, the system may suffer **deadlock** – the whole system stops because two or more components are waiting for each other to continue (and they never do). Also, concurrent activities may wish to alter the same data from memory – if this is not carefully managed, the data may be corrupted rather than correctly updated. This is explained in more detail in a later unit.

SAQ 3

Summarise the main benefits and costs of concurrent systems.

ANSWER.....

The benefits are more efficient use of hardware, increased responsiveness to the user, increased speed of computation (for parallel systems) and better modelling of systems that interact with a concurrent real world

The costs are increased complexity of both hardware and software, and new ways for systems to fail (such as deadlock), as compared to sequential systems.

6.3 Distributed systems

Motivation

We note, in passing, that distributed systems are also concurrent systems – each node may be carrying out processing at the same time as one or more other nodes. However, this section is concerned specifically with the reasons for distribution.

An interesting example of history’s circular paths lies with the original spur to development of early distributed systems. This was the demand for remote access to powerful computing systems, mainly for scientific calculations, and it led to early efforts to link computers at a small number of US universities, eventually growing into what we know as the internet.

In the last few years, the topic of **grid computing** has come to the fore as a way of harnessing the power of many computers distributed over remote sites and communicating via the internet to solve problems, often scientific in nature, requiring very great processing power. This can be seen as turning the original motivation on its head. In early distributed systems, in order to carry out certain tasks it was essential to involve multiple computers. Now, because we have large networks of linked computers available, we can envisage carrying out larger and larger computing tasks – arguably the technology now drives the type of task that may be undertaken.

This is a somewhat specialised motivation for distributed systems – for most users one of the main attractions is the ability to exchange files and messages between remote systems.

The other major application is sharing of resources generally, hardware (such as printers and storage), software (such as search engines) and data (such as web pages). The development of the Web and associated graphical browsers since the mid-1990s has fuelled explosive growth in the internet, in both the number of hosts and the number of users.

Benefits

We have seen that distributed systems facilitate sharing of a wide variety of resources, which in the case of hardware resources may reduce the overall cost of a system. It may also increase the efficiency of a system by locating data and processing power close to where they are most needed (see the supermarket IT example earlier). Compared to a centralised system, this can minimise the traffic on potentially slow, unreliable or expensive communications links.

Distributed systems can be more **fault-tolerant**. For example, the internet often has multiple possible routes for messages to travel – if one route is blocked or faulty, messages can usually be rerouted. Fly-by-wire aircraft normally have multiple

computers linked in a network – if one fails, another one can take over its functions. For example, the Airbus A320 shown in Figure 1 has five main computer systems in a network and, if necessary, the plane can fly with just one working computer.

Such systems are also scalable – we can add or remove hosts to match the demands on the system, usually without the need to interrupt normal operations.

Distributed systems have become more widespread as networked communication has become cheaper and more reliable. Now that microprocessors are also relatively cheap, it is often convenient to design systems that require concurrency by allocating a separate processor for each major function and linking these in a distributed system. This may explain why some current luxury cars may contain over 100 microprocessors.

Costs

If communications links fail or become faulty, some components of the system may become unreachable. Slow links may cause bottlenecks and difficulties in coordinating the system.

Security is potentially more difficult in a distributed system – there are more points of access than in a centralised system and some communication links may use public telephone lines or easily interceptable wireless communication.

Operating systems and other supporting software must be more complex to cope with distribution as well as concurrency. Added to this, distributed systems are often heterogeneous, comprising a wide variety of different hardware or software components. The ability of systems to work across a variety of different environments or platforms is termed **interoperability**.

Maintaining software located on many hosts in a distributed system could be more problematic than on a centralised system; for example, newly issued versions of software components may need to be loaded onto many different and potentially diverse hosts. Again history may be repeating itself with the current growth in **application service providers (ASPs)**, where software runs on centralised servers and is accessed over the internet using a browser or specialised client software. Users do not own the software or run it on local machines.

This resembles the centralised systems and **computer bureau** approach of the 1960s, updated for the internet age. A computer bureau allowed its clients to pay for the computer time needed to run the client's business software, such as payroll or accounting programs, on the bureau's computers. With no internet at that time, data transfer was via magnetic disks or tapes, a much slower and more cumbersome approach.

Finally, distribution introduces new ways in which systems can fail as a result of failures or faults in the individual system components. As we saw earlier, it makes it impossible to completely synchronise the system so that all components share the same clock time.

SAQ 4

Summarise the main benefits and costs of distributed systems.

ANSWER.....

The benefits include sharing of resources, which may be hardware, software or data. Distributed systems also offer advantages in scalability, fault-tolerance, interoperability and, if properly designed, can balance the local processing, centralised processing and communication activities of a system to suit the needs of the users.

The costs are the introduction of new ways for the system to fail – the network connections may fail or become unreliable. Ensuring security may also be more difficult

when there are more points of access as compared to a centralised system. It may also be more complex to maintain possibly different types of software on a number of hosts and to ensure that they show interoperability.

6.4 Mobile systems

Motivation

As computers and other electronic devices have become smaller, it has become both feasible and convenient for many of us to carry devices that have more computing power than machines that filled a room some decades ago. The term **ubiquitous computing** has been used to describe a world in which small highly portable computing devices are everywhere in our environment and integrated with everyday life. Mobile computing clearly applies to systems like mobile phones and laptops but also includes digital cameras, PDAs and no doubt many other devices as ubiquitous computing becomes more ... ubiquitous! The motivation is allowing access to computing facilities or telecommunication from a very wide range of locations.

Benefits

The benefits of mobile phone systems are obvious, and the evidence of over two billion mobile phones in circulation worldwide is that the benefits are widely appreciated. It may be a little less obvious that many mobile phones contain quite powerful processors and can run Java programs, or other software specially written for this environment, in order to carry out business functions, play games and so on. The drawbacks of mobile phones are also clear, for example to anyone who has shared a railway carriage or an airport lounge with insensitive phone users. However, this is more a social than a technical issue and lies outside the scope of this course.

Being able to carry a laptop computer and link it to a local computing environment allows users to get the best from their personal computing facilities and any data stored there, as well as using local facilities such as printers, scanners or file servers. The link to the local system may be wireless (Wi-Fi or other radio links) or can be via cables – it is still a mobile system in either case.

A side benefit of mobile systems that operate wirelessly is to remove the need for fixed wiring, which can be costly and disruptive, although this is actually a benefit of the wireless technology, not of their mobility.

Some mobile systems can take advantage of satellite location services (such as GPS) to become **location-aware**. This may enable them to display locally relevant information or otherwise behave in different ways depending on their physical location.

Exercise 3

It is important to note that wireless systems are not synonymous with mobile systems, although the categories do overlap. Give an example of a wireless system that is not mobile and another example of a wireless system that is mobile. Can a mobile system be ‘not wireless’?

Discussion.....

Wireless links can use, for example, radio, microwave or infra-red radiation. Fixed wireless links are often used to connect computer systems in one building to another building some distance away, perhaps using a microwave or satellite dish on the roof of each building. This is a wireless system that is not mobile.

A mobile phone system has wireless components (the phones or handsets) as well as fixed components for routing the calls once they are received from the handsets.

A laptop computer can be moved around and can connect to a local network either using a wire cable or wirelessly. In either case it is still a mobile part of the system, so a wired laptop is an example of a mobile, non-wireless system.

Costs

Communications links in mobile systems are typically slower, have less capacity and are less reliable than those in fixed systems. Developments in wireless protocols (such as the IEEE 802.11 series) are addressing these limitations and steady improvements have been seen in recent years.

Security in mobile systems can also be a crucial issue – anyone can pick up or send radio signals in an attempt to intercept or perhaps interfere with mobile communications. With lower communications **bandwidth** (i.e. data transfer rate) than fixed systems, the use of encryption to protect communications is more of an overhead. Physical security is also more of a problem – a laptop or a phone containing sensitive information can be accidentally left behind on a train. This may be a more likely source of security breaches than, say, accidentally giving unauthorised people access to a fixed-location system.

SAQ 5

Summarise the main benefits and costs of mobile systems.

ANSWER.....

The benefits are that users can communicate and access the other parts of a distributed system from a wider variety of locations. For example, linking a laptop computer to a local computing environment allows users to get the best of their personal computing facilities and any data stored there as well as using local facilities such as printers, scanners or file servers. For mobile systems that are wireless there are additional benefits of avoiding the cost and disruption of fixed wiring installation.

The costs are mainly, but not entirely, related to wireless mobile systems. There may be an increased security risk in allowing mobile users to link to a system whether wirelessly or not, and mobile components such as laptops are more easily stolen or lost than fixed components. Wireless communications such as radio signals are more vulnerable to interception and often are lower in bandwidth than wired systems. These drawbacks can be overcome, for example by encryption or by better hardware, but these have their own associated costs.

6.5 Enterprise systems

The term **enterprise** is often used to mean a large organisation, such as an industrial company, a bank or a government department. Typically an enterprise will have many employees and a number of component parts, perhaps located at many different sites. It will require many different applications, from financial applications, such as accountancy or ecommerce, through to applications, to support its core business, such as industrial design or multimedia software. The computing systems suitable for the requirements of such organisations are known as **enterprise systems** – the software that runs on these systems is **enterprise software**.

The various characteristics of these enterprises define the typical requirements of enterprise software. To allow reliable, non-stop, large-scale operation, enterprise software must incorporate the following attributes.

- ▶ Availability – being accessible from many locations at all times.
- ▶ Scalability – the capability to deal with highly fluctuating loads on the system.
- ▶ Reliability – maintaining correctness of operations and data even when faults occur.
- ▶ Security – controlling access to the system and its data to prevent possible attacks.

To ensure availability and scalability, enterprise systems must make use of concurrency, distribution and possibly mobile components. However, these same characteristics introduce particular problems in achieving reliability and security. For example, a distributed system typically offers many more points of access than a centralised system, hence giving more potential security problems to guard against. In later units we will consider these issues in more detail, using a case study implemented with the Java Platform, Enterprise Edition (Java EE) software framework.

SAQ 6

Draw a diagram (such as a Venn diagram, showing sets) to show how each of the following categories of computer system is distinct from, includes, or overlaps with the others.

- ▶ Sequential
- ▶ Concurrent
- ▶ Distributed
- ▶ Mobile

If you are not familiar with Venn diagrams, feel free to devise your own way to show this graphically.

ANSWER.....

There may be a number of possible ways to answer this. We take ‘mobile’ to mean a system at least some of whose components are mobile (like a mobile phone system which includes a considerable amount of fixed hardware as well as the mobile handsets).

In this interpretation, the sets for sequential and concurrent systems have no overlap. All distributed systems are (potentially) concurrent, and mobile systems may be distributed or could be independent (like a PDA). These independent mobile systems might or might not be capable of concurrency depending on their hardware and software design, although most such systems are likely to be concurrent nowadays. Our diagram is shown in Figure 11 – the size of the shapes and the size of their overlap is not significant.

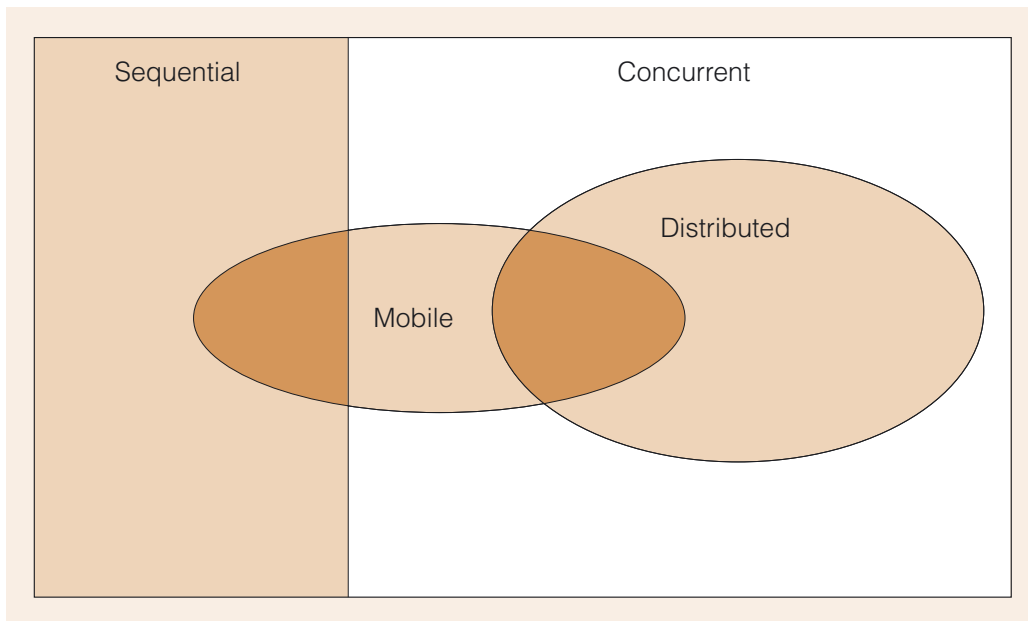


Figure 11 A diagram showing the relationship between different categories of computer system

7

Case study

We now outline a case study that we will use throughout the course to provide a number of scenarios that illustrate issues associated with concurrent and distributed systems. We will develop the case study by considering a series of scenarios of increasing complexity.

The system whose development we aim to discuss is as follows:

Musical Instrument Auction Site

A music shop sells second-hand and rare musical instruments. The instruments are stored in a warehouse from where they can be shipped to customers who have purchased them. There is a server computer in the music shop so staff can keep the catalogue updated. There is also a computer in the warehouse to help keep track of goods in and out and stock currently in the warehouse.

The server in the shop is accessible to the public via the internet. The shop server has to communicate with various financial systems in order to process payments. Customers register with the system before use, entering contact details and financial details so that they can pay for purchases. The payments handling is outsourced to another organisation that uses different enterprise software technology from the music shop systems.

The shop system has software to facilitate auctions over the internet. Instead of a fixed price, each item has a starting price, a fixed date and time at which the auction finishes, and possibly a reserve price (minimum acceptable payment) and a 'buy it now' price. If no bids have been received so far, the customer can secure the item for the 'buy it now' price and no auction takes place.

A customer places bids over the internet, by indicating the highest price they are prepared to pay. The auction system automatically bids for them until it has reached that maximum price. Then it emails the customer to ask if they want to raise their maximum bid. The highest bid at the exact time the auction finishes secures the musical instrument. The system then accepts payment and authorises the warehouse to despatch the instrument.

The auction site can accept bids from mobile devices such as mobile 'smart' phones. It uses streaming technology to convey multimedia information about items for sale.

This system is quite complex, but also quite typical of systems currently in use. It must potentially deal with many customers concurrently, as well as allowing access by staff in the shop or in the warehouse. It is clearly a distributed system, some of whose components are mobile. It will allow us to consider many aspects of the design and implementation of concurrent and distributed systems.

In practice nowadays, the server would probably not be located in the shop. Most businesses now rent server space from a specialist provider, although in the early days of the Web there was a much more 'do-it-yourself' approach. For this case study, we ignore the possibility of renting server space as it does not significantly affect the practical implementation issues we want to consider.

However, we will begin by considering a radically simplified version of this system in the following scenario:

Music Shop – Scenario

A music shop sells new musical instruments ‘over the counter’ and has a separate business in second-hand and rare musical instruments. The owners want to automate this second-hand business, so that eventually it can become an online business and also so that it takes up less time for the sales staff in the shop. A further benefit is that the instruments do not need to be stored in the shop itself, thus freeing up space for the mainstream instrument business.

They start with a single computer system running on a PC in the shop. Customers can use the PC to browse through the available instruments and to reserve instruments that they want to buy. They then pay the shop staff and the instruments are shipped from a warehouse to their home address. The reserved instruments are marked as sold when viewed on the computer screen.

Customers take turns at the single PC on a first-come, first-served basis.

In our practical activities we give the music shop the brand name ‘The Music Store’.



Activity 1.1

Getting started with the Music Shop.



Activity 1.2

Extending the Music Shop example.

8

Summary

In this unit, we have defined what is meant by concurrent, distributed and mobile computer systems and explained why they are important. Concurrency and distribution are both important in modelling and regulating a wide variety of real-world systems that are themselves concurrent and distributed.

Concurrency is also useful in improving the efficient use of computer hardware resources and in ensuring the responsiveness of interactive systems. Implementing concurrency introduces a number of difficulties in programming that must be addressed by appropriate hardware and software design.

Distributed systems aim to share hardware resources as well as various kinds of data. They have advantages of fault tolerance and scalability, but again these aspects can introduce additional problems to those present in centralised systems. Mobile systems are a special, but increasingly important, type of distributed system, with potential difficulties in mobile communications and security being the main costs to offset against their undoubted benefits.

Concurrency and distribution arise in everyday use of typical personal computer systems, even though some users may not be aware of this.

We have also introduced you to the Java IDE you will be using throughout this course and a case study involving an internet auction system for musical instruments. In this unit we considered a greatly simplified system that relates to this case study – we will return to this a number of times in later units, adding increasing complexity at each stage.

In the next unit we will look in more technical detail at how concurrency is implemented on computer systems by means of processes.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ explain what is meant by concurrency and distribution in computer systems;
- ▶ give reasons why concurrent and distributed computer systems are important;
- ▶ be aware of the historical context leading to modern concurrent and distributed systems;
- ▶ give examples of concurrent systems and distributed systems;
- ▶ distinguish between different types of concurrency in computer systems;
- ▶ discuss the respective motivations for concurrent, distributed and mobile systems and their potential benefits and costs;
- ▶ be familiar with the case study that will be used throughout the course to illustrate concepts in more detail;
- ▶ use the Java IDE provided to view sample programs and carry out simple programming activities.

Glossary

application service provider (ASP) An organisation offering access to application software run on centralised servers and accessed over the internet using a browser or specialised client software.

application software Software designed to carry out a function for the user, e.g. a payroll application or a word processing program (as opposed to systems software like the operating system or middleware).

bandwidth The data transfer capacity of a communications link, usually measured for digital communications in units of bits per second.

central processing unit (CPU) See **processor**.

centralised computer system A computer system where all, or almost all, of the processing is carried out at one central location.

computer bureau The clients of a computer bureau pay for the computer time needed to run their business software on the bureau's computers – a similar approach to current application service providers, however this idea predates the internet so data transfer was via magnetic disks or tapes.

concurrent system A system which may have a number of activities active at the same time and allows all of its active activities to make progress.

deadlock A state of a concurrent system where the whole system stops because two or more components are waiting for each other to continue (and they never do).

distributed system A system containing a number of distinct components at different locations, where each component is, in some sense, a computer system itself and the components are linked by a network. The system may demonstrate transparency to varying degrees.

dumb terminal A computer terminal with very little local processing power – simply a screen and keyboard together with some simple communications links.

embedded system A computer system that is physically part of a larger system (often a real-time system) and may monitor or control the system of which it forms a part.

enterprise A large organisation, such as an industrial company, a bank or a government department.

enterprise software Software that runs on enterprise systems.

enterprise system A computing system suitable for the requirements of an enterprise.

fault-tolerant system A system that can continue normally (or perhaps with reduced functionality) after a hardware or software error has occurred.

grid computing A way of harnessing the power of many computers distributed across the internet to solve problems requiring great processing power.

host An individual component of a distributed system that has some processing capability.

interoperability The ability of systems to work across a variety of different environments or platforms.

local area network (LAN) A computer network where all the hosts are physically close together, often in the same building, and normally entirely controlled by one organisation or department (in contrast to a **WAN**).

location-aware system A mobile system that can take advantage of satellite location services (such as GPS) to determine its location and hence adapt its behaviour depending on its physical location.

memory Short-term, rapidly accessed computer hardware for storing data – often, but not always, volatile (loses data if powered off), e.g. **RAM**.

mobile system A distributed computer system some of whose components are not fixed in their location, e.g. a mobile phone network.

monolithic architecture A design of computer system in which all, or almost all, the processing is carried out at one central location, e.g. a mainframe computer.

multi-core A design of CPU chip with more than one processing unit (or 'execution core') that can improve performance by operating in parallel – the special case of two processing units is called dual-core.

multiprocessor A computer system with more than one CPU – this could mean anything from two CPUs to thousands of CPUs.

node See **host**.

operating system Software that interacts with the underlying hardware of a computer system and provides a layer or platform on which other software, such as application software, can run without needing detailed knowledge of the hardware.

parallel bus A set of parallel wiring connections that can link closely positioned CPUs in a multiprocessor system.

parallel system A concurrent system that can really carry out a number of activities simultaneously because it has more than one processor.

personal digital assistant (PDA) A small handheld computing device, usually with relatively limited resources of display, memory and storage.

processor The part of a computer system that carries out the instructions of a software program in order to process data.

pseudo-parallel system A concurrent system which shares processor time between a number of activities but only one activity is actually running at any given time.

random access memory (RAM) Short-term, rapidly accessed hardware for storing data – normally in the form of volatile semiconductor devices (loses data if it loses power).

real-time system A system which must guarantee to respond correctly to inputs within a specified time period.

responsive Acceptably fast to respond to a request for action or information – the definition of acceptability depends on the context.

scalable system A system whose capacity can be increased, or less commonly decreased, to match changes in demand (where demand may be measured in number of users or amount of processing).

scheduling Allocation of processor (i.e. CPU) time between the various activities in a concurrent system.

sequential system A system in which each activity must complete execution before another activity can start.

storage Computer hardware for holding data in the long term (even after being powered down) – typically uses devices such as magnetic or optical disks that are slower but cheaper per unit of data than memory.

TCP/IP TCP (Transmission Control Protocol) and IP (Internet Protocol) are the most commonly used specifications of the detailed format of messages exchanged between internet hosts.

terminal A device for accessing a centralised computer system via local cabling or, if more remote, via the telephone system. Some terminals have limited processing ability, often to aid in graphical display, but the bulk of processing in the system is carried out on the central processor.

time-sharing A rather dated term for a concurrent system with a number of users that allocates time to each user, so that each has the impression that the computer is responding to just their request.

time-slicing A scheduling approach for concurrent systems in which each activity gets a fixed and equal amount of processor time before the processing switches to the next activity.

transparency The extent to which a distributed system appears to its users as a coherent system providing a range of functions (so that the user does not need to know that the system is distributed).

ubiquitous computing A situation where computing devices are so small and cheap that they occur in huge numbers throughout everyday environments.

uniprocessor A computer system with a single central processing unit (CPU).

unresponsive Unacceptably slow to respond to a request for action or information – the definition of acceptability depends on the context.

wide area network (WAN) A computer network that spans a large geographical area such as a region, a country or the world and which typically relies on public providers of telecoms or other long-distance services for part of its communication path (in contrast to a **LAN**).

References

- Bacon, J. (1998) *Concurrent Systems* (2nd edn), Addison-Wesley.
- Burns, A. and Wellings, A. (1998) *Concurrency in Ada* (2nd edn), Cambridge University Press.
- Coulouris, G., Dollimore, J. and Kindberg, T. (2001) *Distributed Systems Concepts and Design* (3rd edn), Addison-Wesley.
- Living Internet (2006) [online] Available from: <http://www.livinginternet.com/e/ei.htm> [Accessed 10 May 2007].
- Sterling, B. (1996) [online] 'A Short History of the Internet', Available from: <http://www.library.yale.edu/div/instruct/internet/history.htm> [Accessed 10 May 2007].

Acknowledgements

Grateful acknowledgement is made to the following sources:

Figure 1: Courtesy Airbus UK Ltd.

Figure 2: Courtesy IBM Corporate Archives.

Figure 7: Professor David May, University of Bristol.

Index

A

application service providers (ASPs) 31

application software 17

ARPANET 13, 21

B

bandwidth 33

C

central processing unit (CPU)
see processor

centralised computer system 12

computer bureau 31

concurrent system 5
benefits 29
costs 29
definition 15
efficiency and service 11
motivation for 9, 12, 27

D

deadlock 29

distributed system 5, 16
benefits 30
characteristics 20–21
communication in 22
costs 31
definition 20
efficiency and service 13
motivation for 9, 13, 30

E

embedded system 10

enterprise 33
software 33
systems 33

Ethernet 22, 25
wireless 22

F

fault-tolerance 30

G

grid computing 25, 30

H

heterogeneous system 24

host 21

I

internet 24
history of 21
interoperability 31

L

local area network (LAN) 16, 22

M

memory 16
mobile system 5
benefits 32
costs 33
location-aware 32
motivation for 32
ubiquitous computing 32

monolithic architecture 12

multi-core technology 18

multiprocessor system 16
communication in 22

Musical Instrument Auction Site 36

N

node 21

O

operating system 17

P

parallel
bus 22
system 15

personal digital assistant (PDA) 6

processor 16
core 18
execution core 18

pseudo-parallel system 15

R

random access memory (RAM) 16

real-time system 10

responsiveness 11

S

scalability 14, 31, 34

scheduling 29

sequential system 15
benefits 27
costs 27
motivation for 26

storage 16

T

TCP/IP protocol 20, 24

terminal 12
dumb 12

time-sharing 12
in chess 13

time-slicing 29

transparency 20

transputer 19

U

ubiquitous computing 32

uniprocessor system 16

unresponsiveness 10

W

wide area network (WAN) 16, 22

wired communication 23

wireless communication 22–23

